

[BOJ] 3273(C++/cpp)

☒ 미완	☐
☒ 언어	C++
≡ 문제 출제	BOJ
✨ 진행 상황	성공
≡ 알고리즘 분류	투 포인터
☒ 복습 필요	☒
📅 풀이날	@2025년 3월 10일
☒ 난도	실버 3
☒ 발전 가능성 여부	☐

문제

백준 3273번 - 두 수의 합

- 주어진 수열에서 두 수를 선택하여 합이 x 가 되는 경우의 수를 구하는 문제입니다.
- 중복 없이 서로 다른 두 수의 합을 찾아야 합니다.

풀이

아이디어

아이디어

- 배열을 정렬하여 투 포인터를 사용할 준비를 합니다.
- 두 개의 포인터 ($left$, $right$)를 사용하여 탐색합니다.
 - $left$ 는 배열의 가장 왼쪽(작은 값), $right$ 는 배열의 가장 오른쪽(큰 값)에서 시작.
 - $nums[left] + nums[right]$ 의 값을 비교하면서 x 와 일치하는 경우를 찾음.
- 조건에 따라 포인터 이동
 - $nums[left] + nums[right] == x \rightarrow$ 정답 카운트 증가 후 $left++$, $right--$

- `nums[left] + nums[right] > X` → `right--` (합을 줄이기 위해 큰 값을 감소)
- `nums[left] + nums[right] < X` → `left++` (합을 키우기 위해 작은 값을 증가)

4. `left < right` 가 성립하는 동안 반복하며 탐색 진행.

코드

```
#include <iostream>
#include <algorithm>
#include <vector>

using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> nums(n);

    for (int i = 0; i < n; i++) {
        cin >> nums[i];
    }

    sort(nums.begin(), nums.end()); // 투 포인터를 위한 정렬

    int x;
    cin >> x;

    int ans = 0;
    int left = 0, right = n - 1;

    while (left < right) {
        int sum = nums[left] + nums[right];

        if (sum == x) {
            ans++;
            left++;
            right--;
        }
    }
}
```

```

else if (sum > x) {
    right--; // 합이 크면 더 작은 값을 선택
}
else { // sum < x
    left++; // 합이 작으면 더 큰 값을 선택
}
}

cout << ans;
return 0;
}

```

해결 과정에서 어려웠던 점

1. 이중 반복문($O(N^2)$)을 사용하면 시간 초과 발생
 - 정렬 후 투 포인터($O(N \log N) + O(N)$)를 사용하여 해결
2. 중복 문제 고려
 - `left++` , `right--` 를 동시에 이동해야 같은 수가 두 번 카운트되는 문제 방지